
Succinct and Fast Tiny Pointer Hash Tables

How should we design a hash table in 2026?

Xilin Tang Yuqi Mai William Kuszmaul Alex Conway
Cornell University Carnegie Mellon University Cornell Tech

VLDB 2026

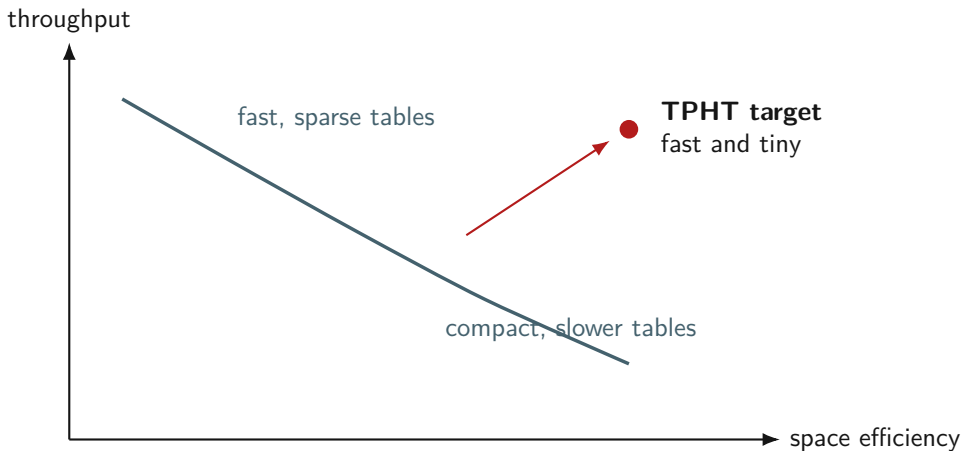
Everyone already knows hash tables matter.

The harder question is: **after decades of tuning**, where is there still room to move?

Modern pressure

- data grows faster than memory budgets
- metadata and slack quietly dominate footprint
- cache misses, not asymptotics, decide latency

Can we make a hash table that is both **near the information bound** and **systems fast**?



Collision resolution pays in empty slots, metadata, or full pointers.

1. Tiny pointers

Replace 64-bit pointers with 8-bit references through a specialized dereference table.

2. Key quotienting

Store only the key bits that are not already implied by the item's location.

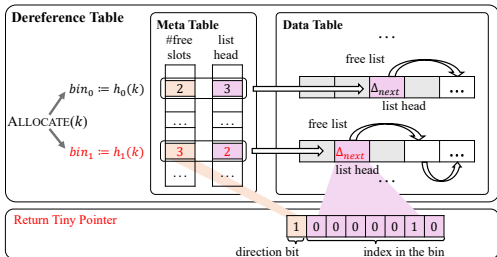
3. Two layouts

Use the same primitives to make either the smallest table or the fastest compact table.



Chained-TPHT 105.4% space efficiency

Flattened-TPHT up to 89.3% higher throughput

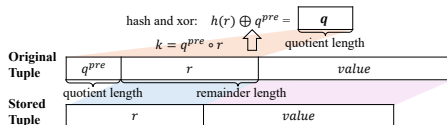


A pointer becomes one byte

- store objects in a fixed-size **dereference table**
- allocate using the **power of two choices**
- encode choice bit + in-bin offset as an 8-bit pointer
- dereference is arithmetic once the owner ID is known

98.2% supported load factor for insertion-only workloads

95% supported load factor with alternating inserts/deletes



The location already stores information

If a key belongs to quotient group q , the high-order information represented by q does not need to be stored again.

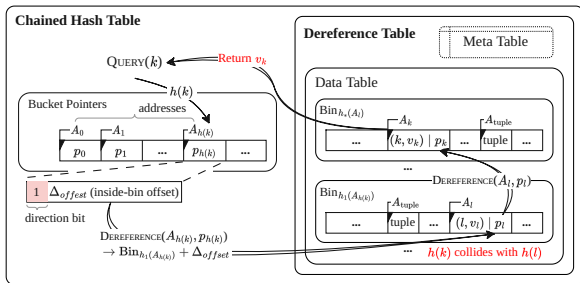
The systems problem

Filters can quotient fingerprints. Hash tables must recover the real key.

TPHT's move

Use a one-round Feistel permutation so the key can be quotiented and still decoded.

$\log n$ bits saved per key, in the useful regime



Make chaining small enough to be modern

- head array stores 1-byte tiny pointers
- nodes store next pointer, quotiented key remainder, value
- quotient groups are exactly chains
- quotienting pays back the pointer overhead

105.4% space efficiency

38.6% mean memory reduction vs. strong baselines

Conventional chaining

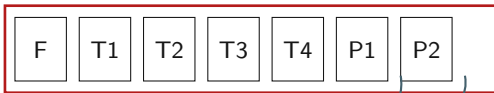
- 8-byte pointers dominate metadata
- head arrays become expensive
- pointer chasing hurts locality

Chained-TPHT

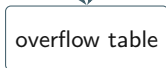
- 1-byte head pointers make many buckets cheap
- compact head array is more cache-resident
- quotienting removes enough key bits to offset links

Chained-TPHT turns the oldest collision-resolution idea into a practical succinct hash table with expected $O(1)$ operations.

64-byte home block



up to four home tuples
plus fingerprints and tiny pointers



Most operations inspect one cache line; overflow tuples cost one extra miss rather than a chain walk.

Spend a little space where latency sees it

- home blocks fit in one cache line
- overflow references are tiny pointers
- SIMD fingerprints prune comparisons
- queries average about 1.25 cache misses

83.4% space efficiency

310.4M average run-phase ops/s

	Chained-TPHT	Flattened-TPHT
Main goal	smallest practical table	fastest compact table
Layout	quotient groups as chains	cache-line home blocks
Critical primitive	tiny pointers everywhere	tiny pointers for overflow
Key trick	quotienting pays for links	home tuples avoid indirection
Best headline	105.4% space efficiency	up to 89.3% faster than baselines

The contribution is not only a point design: it is a reusable recipe for compact, update-friendly tables.

Operations

64-bit keys and values,
inserts, queries, updates,
deletes.

541 ns Flattened-TPHT
99p insert latency during
resizing

Resizing

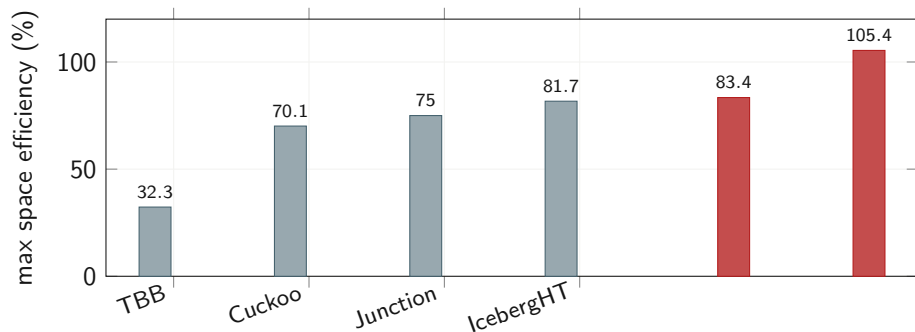
Online resizing avoids global
pauses; Chained-TPHT
staggers memory growth.

778 ns Chained-TPHT
99p insert latency during
resizing

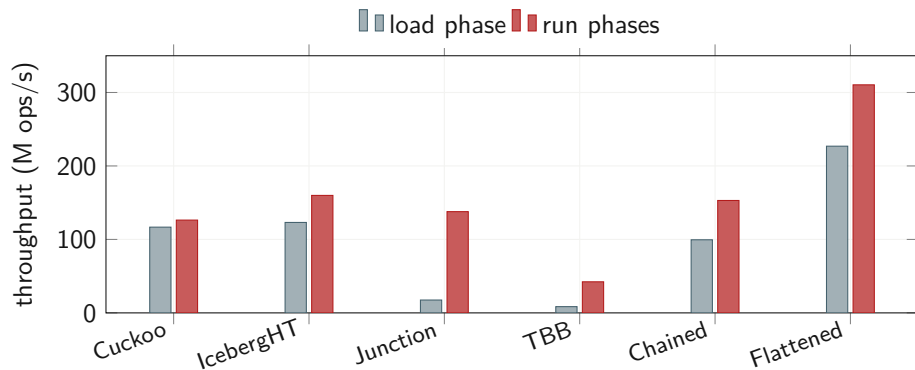
Concurrency

Near-linear scaling to 16
threads in the evaluation.

0.925 Flattened-TPHT
thread scaling factor



Chained-TPHT goes beyond the raw key-value payload size; Flattened-TPHT remains compact while targeting latency.



On YCSB with 16 threads, Flattened-TPHT has the highest average load and run throughput.

1

Tiny pointers make pointer-rich structures worth revisiting.

2

Quotienting turns location into saved key bits without losing recoverability.

3

The same primitives produce both a succinct table and a latency-first table.

TPHT shows that ideas once treated as “too theoretical” can redraw a very practical Pareto frontier.

Thank you.

Questions?

Xilin Tang Cornell University

Artifact: github.com/Xilinion/TinyPtr